

Can an LLM Agent Find the Root Cause of a Microservice Failure?

Progress report: approach, experiments, and results

Yuvraj Sehgal

August 18, 2026

Executive summary

When a microservice system breaks, an engineer looks at monitoring data — metrics, logs, request traces — and works out which component failed and why. We are testing whether an *LLM agent* can do this job, how much monitoring data it actually needs, and whether deep operating-system (kernel) data — which our dataset uniquely provides — makes it better.

To find out, we built a labelled dataset of 95 recorded failures across two demo applications, gave an LLM agent a set of investigation tools over that data, and ran about 650 fully-logged diagnosis experiments. The five main findings:

1. **The agent roughly doubles existing automated methods.** It names the right failing component in 83–87% of incidents, versus about 48% for both a hand-written statistical method and a published state-of-the-art tool — at about one cent of LLM cost per diagnosis.
2. **It degrades gracefully.** Thinning any one data source — keeping only 5% of traces, sampling metrics $12\times$ less often, keeping only error-level logs — has no measurable effect on accuracy. There is no “cliff”.
3. **Very little telemetry is enough.** Applying all the cheap settings at once cuts the data the agent reads by $57\times$ while keeping over 90% of its localization accuracy. Identifying the *type* of fault is what suffers first on a tight data budget.
4. **Kernel data pays in speed and explanations, not raw accuracy.** With good tools over ordinary telemetry, removing kernel data barely moves accuracy — but investigations take a third more steps, and one class of faults (an artificially slowed database) loses its explanation without it. Surprisingly, a *partial* kernel view (raw numbers without interpretation) is worse than none at all.
5. **A library of reusable “experience” helps only if the right entry is chosen.** Pre-written investigation guides fix specific fault types, but a wrong guide misleads more than no guide — selection quality is the missing piece, not the guides themselves.

Every number in this report comes from experiments where the agent could not see anything that gives the answer away, and every diagnosis is fully recorded and machine-checked, so the results can be audited end to end.

1 What this project is about

Modern applications are built from dozens of small services calling each other. When something goes wrong — a database slows down, a service runs out of memory, a co-tenant steals CPU — the symptoms spread: many services get slow at once, and the on-call engineer must work out *which component is the culprit* and *what kind of fault it is*. That activity is called root-cause analysis (RCA).

Engineers do RCA by reading *observability data*: numeric metrics (CPU, memory, network per container), application logs, and distributed traces (records of each user request as it hops between services). Our project adds a fourth, unusual source: *kernel traces* — a complete record from the operating system of every system call, scheduling decision, disk and network event, for every service, with no code changes required. Kernel data can see things the other sources cannot (for example, *why* a process was waiting), but it is enormous and hard to read, which is exactly where an LLM might help.

The project asks four questions:

1. **Capability.** Can an LLM agent, given tools to query this data, match or beat existing automated RCA methods?
2. **Robustness.** How much can the monitoring data be thinned — fewer traces, coarser metrics, filtered logs — before diagnosis quality breaks?
3. **The value of kernel data.** What does the deep kernel layer actually contribute, and in what form is it useful?
4. **Minimum budget.** What is the cheapest monitoring setup that still supports accurate diagnosis? (Observability is expensive to collect and store; if 5% of it is enough, that matters in practice.)

A companion engineering track builds the agent itself as a product-shaped system — with a context builder that prepares an evidence briefing, and a library of reusable investigation guides (“skills”) that anyone can extend — aligned with the industrial architecture our collaborators use.

2 The dataset: 95 recorded failures

We could not study any of this on real production outages (no ground truth), so we built our own corpus, called StrataTrace.

Two test applications, chosen to be different. *Sock Shop* is a small online store (14 services, each with its own database). *Train Ticket* is a much larger booking system (~40 Java services) in which *all* services share one MySQL database — a choke point that request traces cannot see inside, which makes it the perfect stress test for the kernel layer.

Twelve fault types, injected on purpose. For each recording we deliberately broke the system in one known way, in five groups: overloading the whole host (CPU, disk, memory, or network); capping one service’s CPU or memory; application-level faults (an artificially slowed database; a burst of connection errors); dependency faults (freezing a service mid-flight; silently backing up a message queue); and infrastructure faults (a “noisy neighbour” container stealing CPU; degrading one service’s network). Each recording stores the true answer — which component, what fault, when — in a machine-readable label, plus an independent check that the fault actually fired.

What one recording looks like. Each is a four-minute session: one minute healthy, two minutes with the fault active, one minute of recovery — with all four data sources recorded continuously against a single clock (aligned to about a microsecond). In total: 95 labelled recordings, roughly 2.7 TB raw.

Making kernel data usable. The operating-system layer is recorded with LTTng, a low-overhead Linux tracer that timestamps every system call, scheduler decision, disk request and network event. That firehose is 2–13 GB per four-minute recording — far too big to hand to any model — so we derive a ladder of progressively smaller views, each computed mechanically (fixed code, no AI involved) from the level below:

- **L0 — the raw recording.** The complete binary event stream exactly as the tracer wrote it. It is kept archived as provenance; nothing reads it at analysis time except the derivation scripts below.
- **L1 — per-service activity tables.** A script walks the raw stream event by event and attributes each event to the service that caused it, by matching the operating system’s process identifiers against a snapshot — captured with every recording — of which container owned which processes. (This mapping step is what makes the data usable at all on Train Ticket, where every container identifies itself only as “java”.) The attributed events are then aggregated into one row per service per second: typical and worst-case system-call latency, disk and network volume, scheduling activity, and memory-pressure signals. Gigabytes become a few megabytes of tables — the “raw numbers” tier in our experiments.
- **L2 — the wait explanation.** From the scheduler and disk events we reconstruct where each service’s time actually went during the incident: running on a CPU, ready but *waiting its turn* for a CPU (the signature of CPU starvation), waiting for disk, or blocked on something external such as a database or the network. These four shares answer the question activity counters cannot: not “was it slow?” but “what was it waiting *for*?”
- **L3 — the plain-English digest.** A fixed template compares the faulty window against the healthy first minute and writes a few sentences per service (“system-call latency rose to $40\times$ its baseline...”), always with the measured numbers embedded. Because it is generated by rules rather than by a model, it cannot hallucinate.

The agent only ever reads L1–L3; the raw layer stays archived. Later, when we vary “how much kernel data the agent gets,” we are switching these rungs on and off.

Measured collection cost. Recording the kernel layer costs about half a percent of application throughput and $\sim 13\%$ on tail latency at 200 concurrent users — numbers we measured ourselves rather than quoting from folklore.

3 The agent: how a diagnosis happens

The agent is an LLM (GPT-5.4, hosted on Azure) in a loop with seven read-only investigation tools over one recorded incident. It is told only that “an anomaly was detected” in a known time window — nothing about what or where — and it must end by committing to a verdict: *which component* is the root cause, *what kind of fault* it is (from a fixed menu of 13 types), the evidence, and a confidence.

The tools. Each tool compares the healthy baseline window against the incident window and returns a compact summary — never raw data:

- **traces:** per-service request latency percentiles;
- **topology:** which service calls which, and which of those hops slowed down — including calls into components that emit no traces of their own (databases, queues), recovered from the callers’ side;
- **logs:** how each service’s *error rate changed*, plus error messages that are new since the incident began (long-standing noise is flagged as such, so it cannot masquerade as a cause);
- **metrics:** resource signals per container ranked by how much they moved, host-wide signals, and “limit” evidence (a container throttled by its CPU cap, or pinned at a memory ceiling);
- **kernel:** the derived kernel views described above — changed activity, plain-English digests, and the wait explanation;
- **source code:** search and read the application’s own code (useful for checking what a timeout or error message actually does);
- plus a service list for orientation.

The context builder. Before the agent starts, a deterministic component runs the survey pass itself — what changed, where, on every data source — and hands the agent a one-page evidence briefing. This “brief” turned out to be the single best accuracy-per-dollar improvement we made: same or better accuracy at roughly half the tokens and 40% fewer investigation steps.

The skill library. On top of the generic method, we wrote one short investigation guide per fault type — what that problem looks like in evidence, how to confirm it, and how to tell it apart from look-alikes. A selector reads the evidence briefing and either picks the best-matching guide or abstains, in which case the agent proceeds from first principles. Guides are single text files, deliberately written so that anyone can add one for a problem they care about.

Keeping the evaluation honest. Injected-fault datasets can quietly hand the model the answer: recording names that contain the fault type, containers named after the fault, label files. We strip or pseudonymise everything of that kind before the model sees it, and an automated checker rescans every experiment’s full record to certify nothing leaked. This mattered: with the giveaways present, the agent looked far better than it really was (it was effectively reading the answer off identifiers). All numbers in this report are from the honest setup, and every diagnosis ships with its complete, replayable record.

4 How we ran the experiments

Test set. All comparisons use a fixed set of 23 incidents: one per fault type per application (Train Ticket lacks one fault type). The wider corpus of 93 labelled incidents backs the larger baseline sweeps.

What we score. Two things per diagnosis: did the agent name the right component (*localization*), and did it also pick the right fault type from the menu (*fully correct*)? We report both throughout; localization alone is what existing tools can be compared on.

What we compare against. (1) A hand-written statistical method using the same tools; (2) a published, peer-reviewed academic method (multi-source BARO from the RCAEval project) run via its own released code; (3) two trace-only academic methods. All see the same incident windows.

Degrading the data. To ask “how much observability is enough”, we re-run the *same* agent on deterministically thinned copies of the same recordings: keeping 100/50/25/10/5% of traces; sampling metrics every 5/10/30/60 seconds; keeping all logs, warnings-and-up, or errors-only; and giving the kernel layer in full, in reduced forms, or not at all. The agent never changes within a comparison — only the data does.

Repeatability and noise. LLM outputs vary between runs. By re-running identical experiments we measured that any single condition’s headline number carries roughly a ± 9 -point band. We therefore repeat borderline incidents three times, prefer per-fault-type flip evidence over small aggregate deltas, and never quote a difference inside the band as a finding.

Cost. A diagnosis costs about one cent of LLM usage (roughly 25k input tokens, half to two-thirds served from the provider’s cache, and ~ 500 output tokens); a full 23-incident experiment costs \$1–2. The entire experimental program — about 650 diagnoses — cost under \$10.

5 Results

5.1 The agent versus existing methods

Table 1: Share of incidents where each method names the right component.

Method	Right component	Also right fault type
Statistical rule-based method	$\sim 48\%$	38%
Published academic method (BARO)	46%	not applicable
Trace-only academic methods	0–5%	not applicable
Our LLM agent	83–87%	48–57%

The agent roughly doubles both usable baselines on finding the right component (Table 1), at about one cent per diagnosis. Two supporting observations make the case that an *agent* specifically is needed: the two non-LLM methods fail on *different* fault types (so neither approach subsumes the other), and simply feeding kernel data as extra columns into the academic method changes nothing — the value of the deep data is only accessible to something that can reason about it.

5.2 Thinning any one data source: no cliff

We expected accuracy to fall off a cliff somewhere. It does not (Table 2, Fig. 1): across every thinning we tested, accuracy stays inside the measurement-noise band, and 18 of 23 incidents give the *identical* outcome at every trace-thinning level. The reason is structural: the agent’s tools

Table 2: Effect of thinning each data source (23 incidents per setting; “fully correct” = right component and right fault type; measurement noise ± 9 points).

Data source thinned	Range tested	Effect on accuracy
Traces	keep 100% down to 5%	none (flat)
Metrics	every 5 s down to every 60 s	none (flat)
Logs	all lines down to errors only	none (flat)
Kernel	full detail down to absent	−4 points, but +34% more steps

consume each data source as *summaries* (latency percentiles, rate changes, per-second aggregates), and summaries survive sampling; on top of that, most faults leave fingerprints in more than one source, so when one channel thins, the same signature is still visible in another.

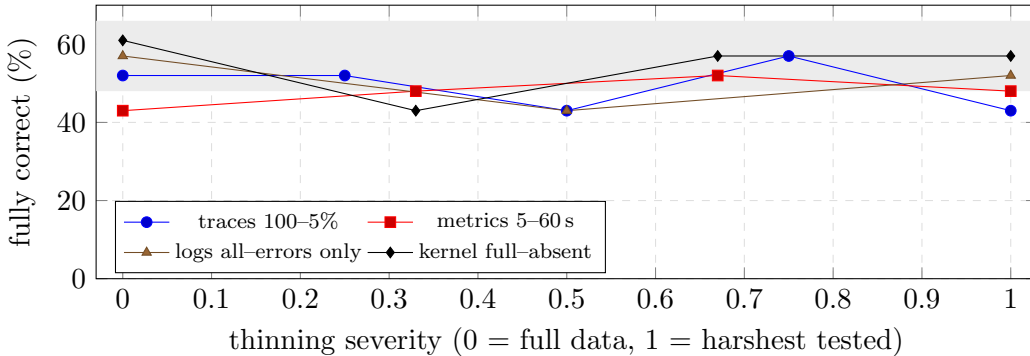


Figure 1: Accuracy under per-source thinning. The shaded band is the measurement noise around the full-data reference; no line leaves it in a consistent direction. The one dip below the band is the “kernel numbers without interpretation” setting discussed next.

5.3 What the kernel layer is actually worth

Table 3: Giving the agent different amounts of kernel data (23 incidents each).

Kernel data provided	Right component	Fully correct	Steps per diagnosis
Everything (numbers, waits, digests)	83%	61%	9.1
Raw numbers only	74%	43%	10.1
English digests only	78%	57%	10.0
No kernel data at all	78%	57%	12.2

Three findings (Table 3):

- **Speed, not rescue.** Removing kernel data entirely costs almost no accuracy — our improved tools over ordinary telemetry cover most faults — but investigations take a third more steps. The kernel’s dependable value is *efficiency*.
- **One fault type genuinely needs it.** The slowed-database fault keeps its *localization* without kernel data but loses its *explanation*: only the kernel’s wait analysis can say “this

database is idle yet all its time is spent waiting on something external — the latency is imposed, not a bug”.

- **A partial view is worse than none.** Handing the model raw per-second kernel numbers without the interpretive layers produced the *worst* result of the whole program — worse than no kernel data at all. Uninterpreted deep data sends the model chasing noise. How you *represent* a data source matters more than whether you have it.

5.4 When a whole data source disappears

Thinning is gentle; removal is not. Taking away *traces entirely* (so the agent cannot see who calls whom at all) is the first change that clearly hurts: accuracy drops 14 points and effort rises 74%, with the damage concentrated on faults whose evidence is the shape of the call graph — mostly in the larger, shared-database application. Two silver linings: first, the slowed-database fault *survives trace removal* in the smaller application, because the kernel’s wait analysis alone carries it — the clearest demonstration that the kernel layer acts as a backup channel when ordinary telemetry is lost. Second, we checked whether kernel data becomes more valuable as traces get *thinner* (rather than absent): it does not — the backup role only activates on outright loss, which is consistent with summaries surviving sampling.

5.5 The minimum observability budget

Table 4: Combining all the cheap settings at once (23 incidents each).

Monitoring setup	Right component	Fully correct	Steps	Data read per incident
Everything (reference)	83–87%	48–57%	9.0	~1.4 GB
Lean: 5% traces, 60 s metrics, errors-only logs	78%	43%	7.6	24 MB (57× less)
Minimal: lean, and no kernel	83%	39%	11.0	12 MB (115× less)

Each cheap setting was individually free; combined (Table 4) they are *almost* free: the lean setup reads 57× less data yet keeps over 90% of the agent’s ability to name the right component. What suffers on a tight budget is naming the exact fault *type* — the diagnosis gets vaguer before it gets wrong. For teams deciding what to collect, the practical reading is: aggressive sampling of traces, metrics and logs is safe for agent-assisted RCA; keep the kernel layer if you care about explanations and investigation speed (it costs only half a percent of throughput to record).

5.6 The reusable-experience (skills) experiment

Table 5: Effect of the investigation-guide library (23 incidents each).

Setup	Right component	Fully correct
No guides (agent from first principles)	83%	57%
With the evidence briefing (our standard setup)	87%	57%
Full guide library available	78%	43%
Library missing the matching guide (unseen fault)	70%	39%

The idea: ship a library of short, human-written investigation guides, one per known problem type, chosen automatically from the evidence. The result is instructive (Table 5): individual guides genuinely help — one fault type is solved *only* when its guide is active, and the guides fixed several fault-type confusions — but the automatic *selection* picks the right guide only about two times in three, and a wrongly-chosen guide misleads more than no guide. When the matching guide is missing entirely (our stand-in for a never-seen-before problem), the selector almost never abstains as designed, and the mismatched guides cost about 18 points versus just working from first principles. The lesson is sharp: *experience libraries are only as good as their selection mechanism*, and that is the component to redesign — the guides themselves, and the agent’s ability to override a bad one, both already work.

5.7 How the agent’s behaviour changes under degradation

Because every step of every diagnosis is recorded, we can also watch *how* the agent works, not just whether it succeeds. Three observations: (1) under thinning, its behaviour barely changes — nothing it relies on was lost; (2) when a source is removed, it compensates broadly, using *all* remaining tools more rather than switching to one substitute; and (3) **effort rises before accuracy falls** — kernel removal costs +34% steps at almost no accuracy loss, trace removal +74% steps before the larger drop. In a real deployment, a rise in investigation effort is therefore an early-warning signal that the monitoring setup has become inadequate — visible without knowing any ground truth. One operational quirk: unless a tool states plainly “this data source is unavailable”, the agent keeps re-querying it service by service, wasting roughly a third of its steps; a one-line tool change fixes this.

5.8 A note on scoring fairness

Some “wrong fault type” verdicts are arguably right: for a burst of injected connection errors, the agent accurately describes connections being killed toward the database but labels it an outage rather than an error storm — a boundary in our menu, not a misunderstanding. Scored with a conservative allowance for four such documented boundary cases (and only when the component is also right), the fully-correct rate rises from 52% to 65%. We report the strict number everywhere and release all evidence text so a reader can judge.

6 What we take away from this

For the research story. The interesting result is not “LLM beats baselines” — it is the shape of the robustness: no cliffs under thinning, real costs only under outright loss or bad representation, and a deep data layer whose value shows up as investigation speed and explanation quality rather than headline accuracy. Those are claims none of the existing automated methods could even express, because they cannot use the kernel layer at all and their accuracy is flat for the uninteresting reason that it is low.

For system builders. Three transferable design lessons: represent deep data before showing it to a model (interpretation beats access); make tools answer definitively when a data source is absent, or the agent wastes a third of its steps; and a deterministic evidence briefing bought more accuracy-per-dollar than any prompt engineering we tried.

For observability practice. Teams over-collect for this use case. Five-percent trace sampling, one-minute metrics and error-only logs kept over 90% of localization ability in our setting; the kernel layer is cheap to record (-0.5% throughput) and repays itself in faster, better-explained diagnoses.

For evaluation practice. Rigour changed our conclusions twice: hiding answer-revealing identifiers collapsed early (inflated) results and forced us to build genuinely better tools; and measuring run-to-run noise stopped us from over-reading single-run differences. Both disciplines — plus full recording of every diagnosis — are what make the numbers in this report trustworthy enough to discuss.

7 Honest limitations

- **One model.** All results use a single LLM (GPT-5.4). The harness supports other providers; replicating on a second model is planned but needs a second API key.
- **Single-run noise.** Condition-level numbers carry a measured ± 9 point band; we repeat borderline incidents and lean on per-fault-type evidence, but any figure we later headline should get dedicated repeats.
- **Two systems, injected faults.** Both subjects are open-source demo applications, and faults are injected rather than organic. The two systems were chosen to differ architecturally, but production diversity is wider.
- **Gentle degradation only.** Our thinning transforms are the kind operators actually use (sampling, coarsening, filtering). Harsher regimes — dropping parts of individual traces, sub-1% retention — remain untested.
- **The fault-type menu.** A fixed 13-label menu forces some verdicts to be “wrong” at label boundaries even when the mechanism is described correctly; we quantify this with a conservative secondary score ($52\% \rightarrow 65\%$).
- **Known unsolved cases.** Two fault types resist every configuration: the silent queue backlog (its evidence barely surfaces in any summary) and the host-network fault’s exact type (its symptoms mimic disk trouble). Both are documented targets rather than hidden failures.

8 What we propose to do next

1. **Redesign the guide selector** — the one component the data clearly indicts. We can iterate on it cheaply by replaying the 200+ recorded evidence briefings through candidate selectors without re-running any diagnoses.
2. **Repeats for headline figures.** Whichever numbers we promote to headline claims get three-run repeats to tighten the noise band.
3. **Cross-model replication** once a second provider key is available (one afternoon, a few dollars).

4. **Write-up decisions.** Whether this becomes one study paper or a dataset paper plus a study paper, and for which venue — the dataset-track deadline in early November is the anchor. The pre-registered fault predictions need their amendment log updated with the outcomes reported here.
5. **Product track** (with our industrial collaborators’ architecture): richer selection for the guide library, mining new guides automatically from recorded diagnoses, and exposing the system behind the interface layer their ecosystem expects.

All raw materials for discussion — every diagnosis record, the analysis scripts, and the per-experiment archives — are stored durably on the cluster and are reproducible end to end.

Appendix: outcome per fault type across key setups

● = component and fault type both right; ○ = right component only; – = miss. Columns: our standard setup (with the evidence briefing); kernel given in full / as raw numbers only / absent; traces removed entirely; the lean and minimal budget setups. S = Sock Shop, T = Train Ticket. Generated directly from the recorded results by the released analysis scripts.

Fault (application)	standard	kernel full	kernel raw	no kernel	no traces	lean	minimal
Host CPU overload (S)	●	●	●	●	●	●	●
Host disk overload (S)	●	●	●	●	●	●	●
Host memory exhaustion (S)	●	●	●	●	●	○	○
Host network degradation (S)	–	–	–	–	–	–	–
Frozen dependency (S)	●	●	●	●	●	●	●
Error burst (S)	○	○	○	○	○	○	○
Noisy neighbour (S)	○	●	○	●	○	○	●
Silent queue backlog (S)	–	–	–	–	–	○	–
Slow database (S)	●	●	●	○	●	●	○
Service CPU limit (S)	●	●	●	●	●	●	●
Service memory cap (S)	○	●	○	●	○	–	–
Single-service network fault (S)	○	○	–	○	–	○	○
Host CPU overload (T)	●	●	●	●	●	●	●
Host disk overload (T)	●	●	●	●	●	●	●
Host memory exhaustion (T)	●	●	●	●	●	●	●
Host network degradation (T)	○	○	○	–	○	○	○
Frozen dependency (T)	●	●	–	●	–	–	–
Error burst (T)	○	○	○	○	○	○	○
Noisy neighbour (T)	○	○	○	○	●	●	○
Slow database (T)	–	–	–	–	–	●	○
Service CPU limit (T)	●	●	●	●	–	–	●
Service memory cap (T)	○	–	○	–	○	○	○
Single-service network fault (T)	–	●	–	●	–	–	○